Classical planning & TAMP

Read the task model. Connect it to robot motion.

Part 1 · Read PDDL

Facts → actions → a task plan

Part 2 · Connect motion

Geometry → feedback → TAMP

30 min presentation · 10 min demo · 5 min recap

Part 1 · Classical planning & PDDL

What should the robot do, and in what order?

Read facts. Understand actions. Predict a plan.

State → **Action** → **Goal**

20 minutes · read the demo PDDL files

What is classical planning?

Find an action sequence that makes the goal true.

Initial state

Robot at delivery; parcel at pickup.

→

Actions

Move, pick, move, place.

→

Goal

Parcel at delivery.

Known state · deterministic outcomes · discrete actions

States, predicates, and facts

A state is the collection of facts that are true now.

Predicate declaration · domain.pddl

```
(robot-at ?l - location)
```

A pattern with a location placeholder.

Concrete fact · problem.pddl

```
(robot-at delivery)
```

A statement about one actual location.

Goals and the closed-world model

Say what must be true; let the planner choose the actions.

Goal excerpt · problem.pddl

```
(:goal (package-at parcel delivery))
```

Initially true

Robot at delivery; hand empty.

Initially false

Holding parcel; parcel at delivery.

PDDL: domain versus problem

Planning Domain Definition Language describes the model.

domain.pddl

Vocabulary
Action rules
Reusable model

problem.pddl

Objects
Initial facts + goal
One delivery instance

PDDL describes the task. Pyperplan searches for a plan.

Objects, types, and variables

Read the names and placeholders in the demo files.

Types excerpt · domain.pddl

```
(:types location package)
```

Objects excerpt · problem.pddl

```
(:objects pickup delivery
          upper-left upper-right - location
          parcel - package)
```

parcel: a package object

?p: variable
-: type label

Read the initial state and goal

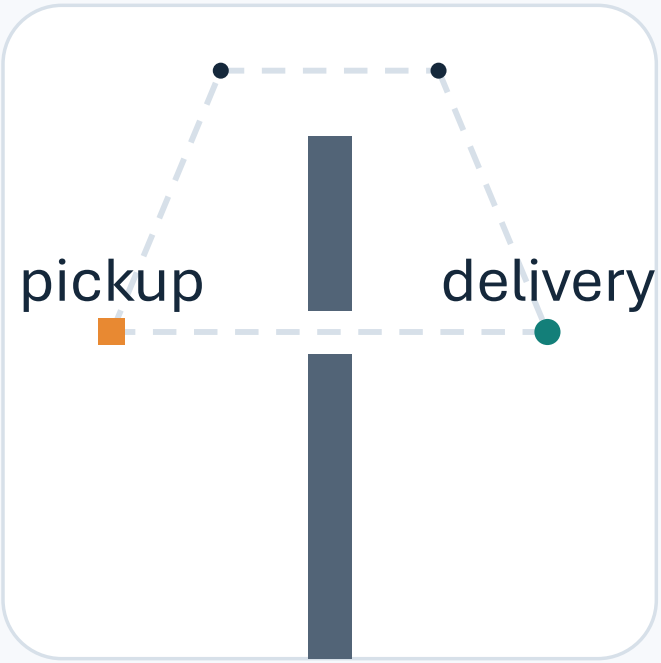
Connect symbolic facts to the delivery scene.

Initial facts · opening excerpt

```
(:init
  (robot-at delivery)
  (package-at parcel pickup)
  (hand-empty)
```

Goal excerpt

```
(:goal (package-at parcel delivery))
```



Anatomy of an action

Parameters identify objects; conditions and effects define behavior.

Parameters

Which package and location?

Preconditions

What must be true before?

Effects

What changes afterward?

move-empty **move-loaded** **pick** **place**

When can we pick?

All three preconditions must hold before pickup.

pick excerpt · domain.pddl

```
(:action pick
  :parameters (?p - package ?l - location)
  :precondition (and
    (robot-at ?l)
    (package-at ?p ?l)
    (hand-empty))
```

Robot and parcel together.

Hand empty.

and = all three

What changes after picking?

Add some facts, remove others, preserve everything else.

pick effects excerpt · domain.pddl

```
:effect (and
  (holding ?p)
  (not (package-at ?p ?l))
  (not (hand-empty)))
```

Before pick · at pickup

```
(package-at parcel pickup)
(hand-empty)
```

After pick

```
(holding parcel)
kept: (robot-at pickup)
```

Read a loaded move

The robot changes location while possession stays true.

move-loaded · preconditions

```
:precondition (and
  (robot-at ?from)
  (holding ?p)
  (can-loaded ?from ?to))
```

Same action · effects

```
:effect (and
  (not (robot-at ?from))
  (robot-at ?to))
```

Holding persists; only the robot location changes.

From variables to a concrete action

Grounding fills placeholders; applicability checks the current facts.

Schema pattern

```
(pick ?p ?l)
```

→

Grounded instance

```
(pick parcel pickup)
```

Initially

Exists, but is not applicable.

After the empty move

Its preconditions now hold.

Search symbolic states

BFS chooses a shortest action sequence in this task model.

```
1  (move-empty delivery pickup)
2  (pick parcel pickup)
3  (move-loaded parcel pickup delivery)
4  (place parcel delivery)
```

Initial > At pickup > Holding > At delivery > Delivered

Part 2 · Task and motion planning

Can the symbolic decisions be realized by robot motion?

Task and Motion Planning

Symbolic task

What should happen?

Geometric motion

Can it happen in this scene?

What does TAMP connect?

Task decisions and geometric feasibility inform each other.

Task level

Named locations
Possession + actions



Motion level

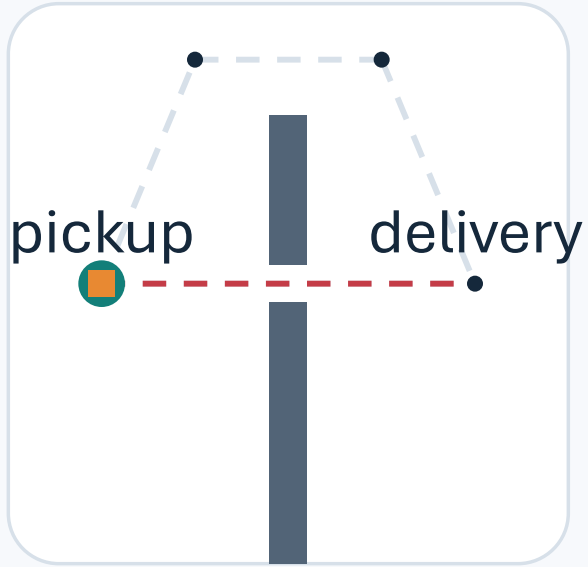
Coordinates + footprint
Collision-free motion

PRM can support motion queries. This demo checks straight segments.

A symbolic plan can fail geometrically

A permitted connection is not a collision certificate.

```
1  (move-empty delivery pickup)
2  (pick parcel pickup)
3  (move-loaded parcel pickup delivery)
4  (place parcel delivery)
```

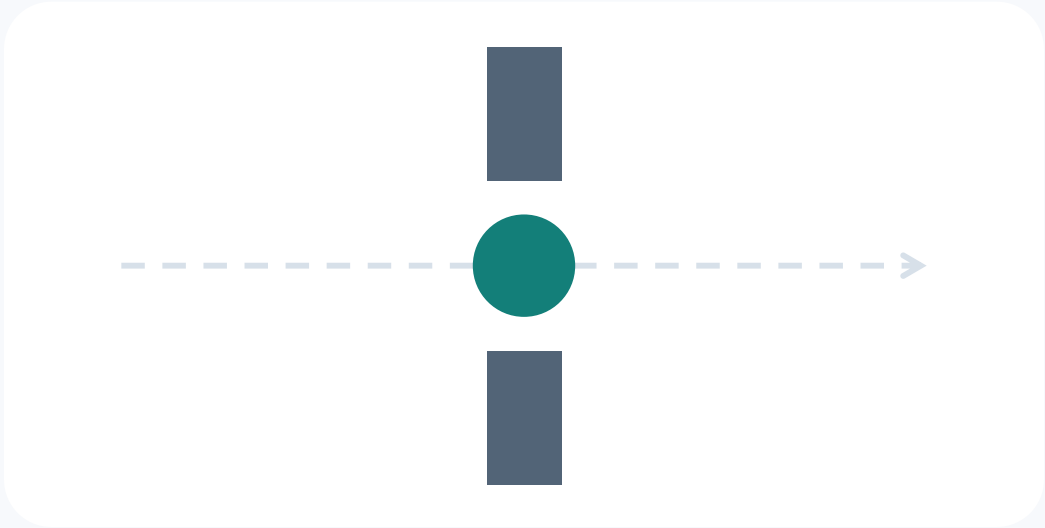


Candidate only · not executed

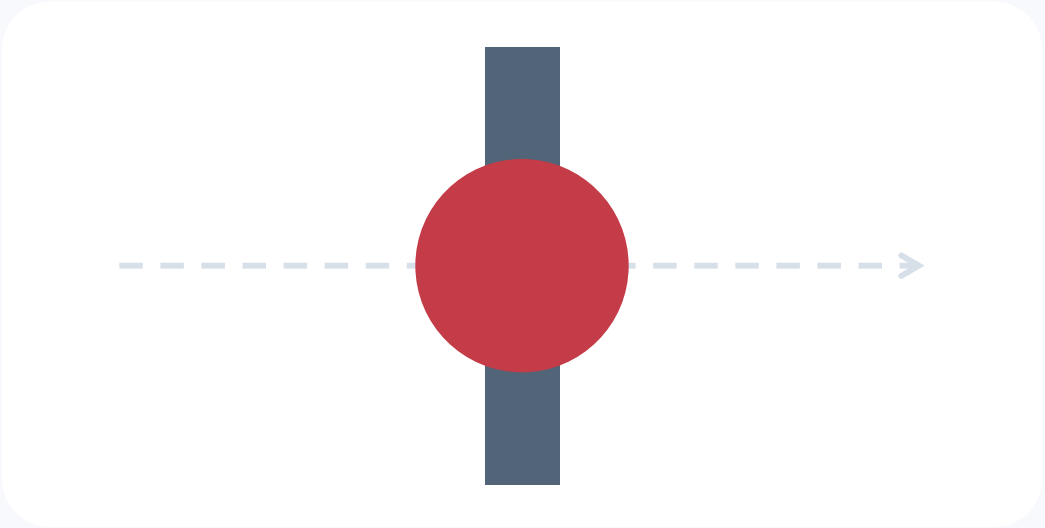
Third action fails geometric validation.

Empty fits; loaded does not

The footprint changes when the robot holds the parcel.



Empty · diameter 0.24 m

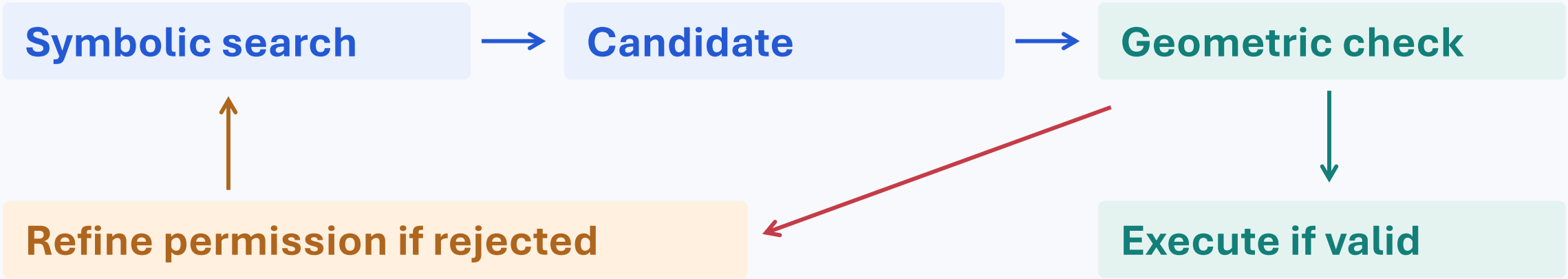


Loaded · diameter 0.50 m

Gap: 0.40 m | Centerline clearance: 0.20 m

The task-and-motion loop

Use geometric feedback to refine the symbolic candidate.



Hypothetical checking; replan from the original initial state.

Refine precisely

Reject one directed connection for one load state.

REMOVE · one loaded direction

```
(can-loaded pickup delivery)
```

KEEP · empty shortcut

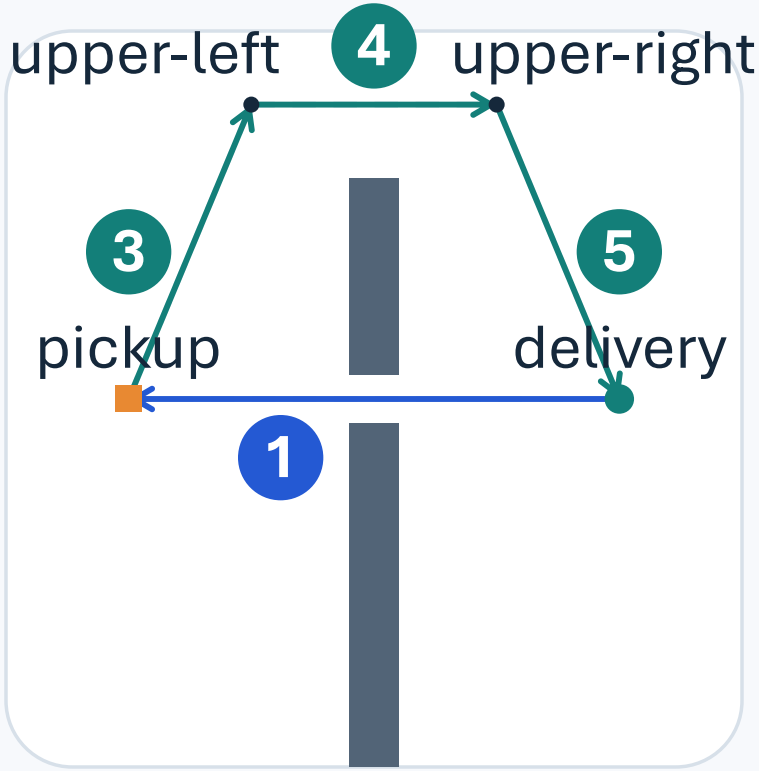
```
(can-empty pickup delivery)
```

KEEP · reverse permission

```
(can-loaded delivery pickup)
```

The revised six-action plan

Keep the empty shortcut; carry the parcel along the upper route.




```
1 (move-empty delivery pickup)
2 (pick parcel pickup)
3 (move-loaded parcel pickup
   upper-left)
4 (move-loaded parcel upper-left
   upper-right)
5 (move-loaded parcel upper-right
   delivery)
6 (place parcel delivery)
```

Planning, validation, and execution

A candidate, a checked plan, and an executed result are different.

Plan

Propose actions from symbolic facts.

Validate

Check motions in a hypothetical state.

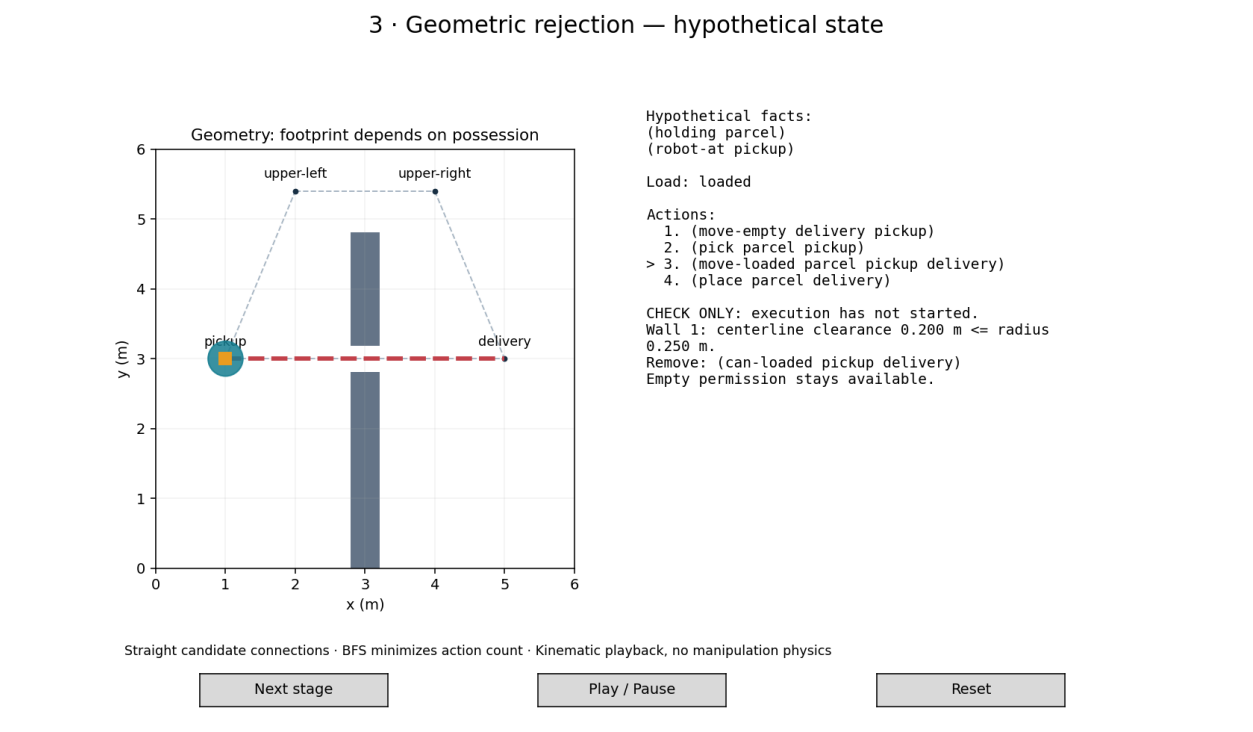
Execute

Animate only the validated plan.

Facts update when an action finishes.

Demo: predict, reject, revise, execute

Watch the facts, footprint, and action sequence together.



Predict the failure.

Inspect the refinement.

10 min demo + 5 min recap

Reading place

The parcel becomes located again when it is released.

Complete action · domain.pddl

```
(:action place
  :parameters (?p - package ?l - location)
  :precondition (and (robot-at ?l) (holding ?p))
  :effect (and
    (package-at ?p ?l)
    (hand-empty)
    (not (holding ?p))))
```

OPTIONAL / DEEPER DISCUSSION

State updates and symbolic search

Ground the action, check its conditions, then update the facts.

$$S_{next} = (S \setminus Delete) \cup Add$$

$$Preconditions \subseteq S$$

Grounding

Substitute concrete objects.

BFS

Search by action count.

OPTIONAL / DEEPER DISCUSSION

Swept-disk geometry

Check the entire designated straight motion against each wall.

$$\min_{s \in [0,1]} \text{distance}((1-s)a + sb, \text{wall}) > \text{radius}$$

Shortcut: 0.20 m clearance

Empty radius: 0.12 m

Passes.

Loaded radius: 0.25 m

Fails; contact also fails.

OPTIONAL / DEEPER DISCUSSION

Failure meanings and broader TAMP

A finite task-and-motion model has specific limits.

Symbolic failure

No plan with the remaining connections.

Geometric failure

A candidate footprint or motion is blocked.

Broader TAMP

Search grasps, poses, placements, and paths.

16 permissions → at most 17 searches in this finite demo.